

TMOP Mesh Optimization in MFEM — capability survey + sliver-repair play-book

Scope: (1) what MFEM's TMOP gives you and how to drive it for meshing; (2) whether/how it can fix the sliver tets in meshing/results/msh/safs_fault_box_nwcut_1000m_lcfar3000.msh, which the 2026-05-20 post-event debug tied to a spurious north-tip slip-rate blow-up.

Audience: a computational scientist who knows the SAFS physics but has not read the TMOP source. Code refs are to this repo (fem/tmop.{hpp,cpp}, miniapps/meshing/). Read alongside PLAN_mesh_quality.md (the gmsh-side attack already shipped) and the project CLAUDE.md sliver notes.

0. TL;DR (the summary you asked for)

- TMOP = Target-Matrix Optimization Paradigm: a variational r-adaptivity engine. It minimizes $F(x) = \sum_T \int_T \mu(J(x)) dx$ over the node positions x , where J is the Jacobian from an ideal/target element to the physical element and μ is a mesh-quality metric (shape / size / alignment). It is a smoother + untangler, driven by an inexact Newton solver that is built to avoid producing inverted ($\det J \leq 0$) elements. Driver: miniapps/meshing/mesh-optimizer.cpp (serial), pmesh-optimizer.cpp (MPI). Inspector: mesh-quality.cpp.
 - What it can do for your mesh: reposition nodes to raise the worst element quality (shape metric 303/302 + worst-case -wctype 2), or untangle genuinely inverted cells (-mid 313). With surface fitting it can let fault-trace nodes slide tangentially along the fault and free surface while staying on them, which is exactly the degree of freedom a surface-trace sliver needs.
 - What it cannot do: it never changes connectivity. No edge swaps, no edge collapse, no node insertion/deletion. A sliver whose 4 vertices are pinned (e.g. all sitting on the fault/free-surface trace) cannot be removed by node motion alone — that needs topology surgery (gmsh Netgen, already wired in PLAN_mesh_quality.md Phase 3) or a re-mesh. TMOP and Netgen are complementary, not substitutes.
 - The decisive gotcha for this mesh: the fault (Physical Surface 101) is an interior surface that MFEM stores as boundary elements (the SAFS WaveOperator finds it via bc_.fault_attr). The free surface (102), bottom (103) and sides (104) are likewise boundary-attributed. mesh-optimizer's built-in boundary handling only understands attributes 1/2/3/4 — it will treat tags 101–104 as free-to-move boundary nodes under the default -bnd. Run naively and TMOP will happily distort the San Andreas fault geometry and lift the free surface off $z=0$. You must either remap attributes, use -fix-bnd, or use surface fitting (details in §6, §7).
 - Honest bottom line: TMOP is worth trying as a polishing pass on the handful of residual near-surface slivers ($\eta \approx 0.03\text{--}0.09$) that PLAN_mesh_quality.md accepted, but because those slivers live on the constrained fault/free-surface trace, the realistic win comes from surface fitting (tangential relaxation), not from the vanilla -fix-bnd smooth. If the slivers are topological, the robust fix remains geometric headroom + gmsh Netgen (already done) or local re-meshing — not TMOP.
-

1. What TMOP is (the math)

TMOP solves the unconstrained minimization

$$\min_x F(x) = \sum_{\text{elements}} \int_{\{\text{target } T\}} \mu(J(x)) dW$$

- $J = \partial x / \partial \bar{x}$ is the Jacobian of the map from the target element W (the shape/size you want) to the physical element (what you have). TMOP factors $A = J \cdot W^{-1}$ so the metric sees only the deviation from the target.
- $\mu(J)$ is a quality metric (TMOP_QualityMetric): a smooth scalar that is minimized when J matches the ideal. Metrics are tagged by what they control: shape (angles/condition number), size ($\det J$), or combinations, in 2D ($\mu_2, \mu_7, \dots$) and 3D ($\mu_{30x}, \mu_{31x}, \mu_{32x}$).
- The target is supplied by a TargetConstructor ($-tid$): ideal shape with unit / equal / initial / analytic / discrete size.
- Minimization is by TMOPNewtonSolver (an inexact Newton / optionally L-BFGS), with an inner Krylov solve (MINRES/CG) of the metric Hessian per step. The line search rejects steps that would invert an element — that is what makes TMOP an untangler rather than just a smoother.

Because everything is variational, μ provides EvalW (energy), EvalP (first derivative / “stress”) and AssembleH (Hessian) — see e.g. TMOP_Metric_303 at fem/tmop.hpp:835.

TMOP operates on an H1 nodal grid function (mesh→SetNodalGridFunction). For a P1 tet mesh (your case) the “nodes” are just the tet vertices, so TMOP is moving vertices. For curved (high-order) meshes it moves the high-order control nodes too.

2. Architecture / key classes

Class	File	Role
TMOP_QualityMetric	fem/tmop.hpp:27	Base; defines EvalW/EvalP/AssembleH.
TMOP_Metric_3xx	fem/tmop.hpp:793+	3D metrics (tets/hexes). See §4.
TMOP_Combo_QualityMetric	fem/tmop.hpp:115	Weighted sum of metrics (shape+size).
TMOP_WorstCaseUntangleOptimizer_Metric	fem/tmop.hpp:200	Wraps a metric to optimize the worst element (β -barrier or p-mean) and/or untangle.
TargetConstructor	fem/tmop.hpp:1577	Builds target W (ideal shape $\times$ chosen size).
TMOP_Integrator	fem/tmop.hpp:1993	The $\int \mu(J)$ nonlinear form integrator; owns limiting, surface fitting, normalization.
TMOPComboIntegrator	fem/tmop_tools	Sum of integrators (e.g. two metrics, two targets).
TMOPNewtonSolver	fem/tmop_tools.hpp	Inverted-element-aware Newton/L-BFGS driver; tracks $\min \det(T)$ for untangling.
TMOPHRSolver	fem/tmop_tools.hpp	Wraps the Newton solver with optional h- and r-adaptivity loop.

Data flow in the driver (mesh-optimizer.cpp):

```
main()
├─ read Mesh, SetNodalFESpace (H1, order = -o)      # nodes become the unknown x
├─ pick TMOP_QualityMetric      (switch on -mid)    # tmop.cpp ll.463-515
├─ pick TargetConstructor      (switch on -tid)     # ll.605-...
├─ [opt] wrap in WorstCaseUntangleOptimizer_Metric # -btype / -wctype, ll.538-579
├─ TMOP_Integrator(metric, target)
└─   └─ SetIntegrationRules(-qt,-qo)
```

```

|   |─ [opt] EnableLimiting(x0, dist, lc)           # -lc: tether to original pos
|   |─ [opt] EnableSurfaceFitting(level_set, marker) # slide nodes on a surface
|   └─ [opt] EnableNormalization(x0)              # -nor: make terms unitless
└─ NonlinearForm a(H1); a.AddDomainIntegrator(integ)
└─ a.SetEssentialVDofs(...) # boundary fixing (ll.1024-1078) <-- the gotcha
└─ TMOPNewtonSolver + TMOPHRSolver.Mult()         # the actual optimization
└─ write optimized.mesh                           # NOTE: MFEM format, not .msh

```

3. How to use it for meshing (driver knobs)

Build (needs conda activate mfem-dev):

```

cd miniapps/meshing
make mesh-optimizer      # serial
make pmesh-optimizer     # parallel (MPI) – use this for the 1.3M-tet SAFS mesh
make mesh-quality        # quality inspector

```

The flags you actually care about (mesh-optimizer.cpp:166–318):

Flag	Meaning	Note for SAFS
-m	input mesh	accepts gmsh v2.2 .msh or MFEM mesh format
-o	nodal FE order	1 for your P1 tets (don't curve them)
-mid	quality metric id	3D ⇒ use a 3xx metric (see §4)
-tid	target type	3 = ideal shape, initial size for graded meshes (§5)
-wctype	worst-case type: 0/1/2 = None/ β /PMean	2 to attack the worst slivers
-btype	barrier: 0/1/2 = None/Shifted/Pseudo	for shifted-barrier untangling (metrics 4/14/66 only)
-bnd / -fix-bnd	move vs fix all boundary nodes	-fix-bnd pins all surfaces (§6)
-lc	limiting constant	tether nodes near original position (geometry-preserving)
-nor	normalization	on — your coords are UTM (~3.8e6 m); makes terms unitless
-ni,-li	Newton / linear iters	untangling wants more (-ni 50 -li 50)
-rtol	Newton rel-tol	1e-5 typical for untangling
-qo	quadrature order	4–8
-fd	finite-diff derivatives	needed by some untangling metrics
-vl	verbosity	1 to watch Newton residual + min det(T)
-d cuda, -pa	device / partial assembly	optional perf

Canonical 3D untangling example shipped in the header (mesh-optimizer.cpp:109):

```
mesh-optimizer -m cube-holes-inv.mesh -o 3 -mid 313 -tid 1 -rtol 1e-5 -li 50 -qo 4 -fd -vl 1
```

mesh-quality (read-only, prints min size / worst aspect-ratio / skew and can dump a VisIt field) is the right tool to measure before/after:

mesh-quality -m <mesh> -size -aspr -skew -visit

4. The 3D metrics that matter here

From the -mid help and fem/tmop.hpp:

id	μ (W form)	Type	When to use
301	$ J J^{-1} / 3 - 1$	3D shape	gentle shape clean-up
302	$ J ^2 J^{-1} ^2 / 9 - 1$	3D shape, polyconvex	robust shape; good worst-case base
303	$ J ^2 / 3 / \det(J)^{2/3} - 1$	3D shape	the standard 3D shape metric
304	$ J ^3 / 3^{3/2} / \det(J) - 1$	3D shape	
313	$ J ^2 \cdot (\tau - \tau_0)^{-2/3} / 3$	3D untangling of 303	inverted cells ($\det J \leq 0$); tracks min det
315	$(\det J - 1)^2$	3D size	size control
316	$\frac{1}{2}(\sqrt{\tau} - 1/\sqrt{\tau})^2$	3D size	size, barrier in det J
321	$ J - J^{-t} ^2$	3D shape+size	shape and size together
328,332,333,334,347	combos	3D shape+size	balanced shape+size (-bec auto-weights)

The worst-case wrapper (TMOP_WorstCaseUntangleOptimizer_Metric, fem/tmop.hpp:186–271) transforms a base metric $\hat{\mu}$:

$$\begin{aligned} \tilde{\mu} &= \hat{\mu} && (\text{WorstCaseType=None}) \\ &= \hat{\mu} / (\beta - \hat{\mu}) && (\text{Beta}) \quad - \text{barrier as } \hat{\mu} \rightarrow \beta \\ &= \hat{\mu}^p && (\text{PMean}) \quad - \text{emphasizes large-}\hat{\mu} \text{ (bad) elements} \end{aligned}$$

This is the key idea for a mesh that is 99.99% fine with a few slivers: minimizing the mean energy barely moves on 762 bad cells out of 1.3M, but the p-mean / worst-case objective concentrates the optimization on the worst tets. Driven via -wctype 2 (PMean) or -wctype 1 (Beta).

min det(J) of the input is computed at mesh-optimizer.cpp:966–999; if it is negative the driver aborts unless you chose an untangling metric (22/311/313/352) or a barrier — so the metric choice must match whether your slivers are merely bad (positive volume, small η) or inverted ($\det J \leq 0$).

For SAFS slivers: check_mesh_quality.py reports $\eta \approx 0.03\text{--}0.09 > 0$, i.e. positive volume, not inverted. So the primary tool is a shape metric + worst-case PMean (e.g. -mid 303 -wctype 2), not 313. Use 313 only if mesh-optimizer reports min det(J) < 0 on the actual file.

5. Targets — preserve the grading (-tid 3)

The SAFS meshes are deliberately graded (fine near the fault corridor, lc_far = 3000 m away). The target type controls what “ideal size” means:

- -tid 1 (IDEAL_SHAPE_UNIT_SIZE): pushes every tet toward unit volume → would try to equalize sizes and destroy your grading.
- -tid 2 (IDEAL_SHAPE_EQUAL_SIZE): equalizes to the mean → also wrong here.
- -tid 3 (IDEAL_SHAPE_INITIAL**_SIZE): keeps each element’s current size, optimizes only shape.**
← correct for shape-only sliver repair on a graded mesh.

- `-tid 4/5` analytic/discrete size fields — overkill unless you want to impose a new size field (then feed the gmsh size field / a background mesh).

So the conservative, grading-preserving recipe is a shape metric with initial-size target.

6. The boundary-fixing gotcha (read this twice)

mesh-optimizer's boundary logic (mesh-optimizer.cpp:1024-1078):

- `-fix-bnd` (move_bnd=false) → `SetEssentialBC(ess_bdr=1)` on all boundary attributes ⇒ every boundary node is fully pinned; only interior bulk nodes move.
- `-bnd` (default, move_bnd=true) → fixes a node component based on its boundary attribute: attr 1 fixes x, 2 fixes y, 3 fixes z, 4 fixes all; any other attribute ⇒ the node is free to move in all directions.

Your mesh's boundary attributes are the gmsh physical tags: fault=101, top=102, bottom=103, sides=104. None are in {1,2,3,4}. Therefore:

- Default `-bnd` would treat the fault, free surface, sides and bottom as free → TMOP distorts the fault geometry and lifts mesh_zmax off 0. Violates two hard invariants in `PLAN_mesh_quality.md` (free-surface $z=0$; fault embedded). Do not do this.
- `-fix-bnd` pins all of them, including the fault and free surface. Safe for the invariants, but it also pins the sliver vertices that live on those surfaces, so only their interior neighbors can relax. Limited upside for surface-trace slivers.

To do it right you have two options:

1. Remap attributes before optimizing so the box behaves physically: free surface (102) → attr 3 (fix z, slide in x,y), sides (104) → attr 1 or 2 (fix the normal component), bottom (103) → attr 3, and decide what to do with the fault (101). This lets surface nodes slide within their plane — already a big improvement for slivers — without leaving the surface.
2. Surface fitting (§7) for the curved fault, where “fix a component” is not enough because the fault is not axis-aligned.

Note: MFEM stores the fault as boundary elements even though it is geometrically interior (each fault triangle is shared by two tets). This is exactly how the SAFS WaveOperator embeds the fault (bc_.fault_attr, wave_operator.hpp). It also means TMOP sees the fault as a constrainable boundary set — good — but the built-in `-bnd` mapping can't keep it on the curved fault surface, so use fitting or fix it fully.

7. Surface fitting = tangential relaxation (the real lever for trace slivers)

TMOP_Integrator::EnableSurfaceFitting (fem/tmop.hpp:2371-2449, miniapp pmesh-fitting.cpp) adds a penalty $\int c \cdot \tilde{s}(x)^2$ that lets a marked node move only along the zero level set of a function s :

- Give it a level-set s_0 whose zero set is the fault surface (a signed distance to the San Andreas surface — you already have the .ts/.stl and a signed-distance capability in the velocity/stress sidecar pipeline), mark the fault DOFs, and TMOP will slide them tangentially on the fault while the shape metric improves the surrounding tets. The fault geometry is preserved to the level-set tolerance; the embedding (2-tet sharing) is preserved because no connectivity changes.
- Do the same with $s = z$ for the free surface (its zero set is $z=0$) to let surface nodes slide in x,y but never leave $z=0$.
- `EnableSurfaceFittingFromSource` (parallel) fits to a level set defined on a finer background mesh — useful if you want sub-element-accurate adherence to the fault STL.

This is the only TMOP mode that gives a surface-trace sliver a real chance: its vertices get to move along the two constraint surfaces (and their intersection line), which is precisely the freedom a flat tet at the faultfree trace needs to fatten up — without violating either invariant.

8. Concrete playbook for **safs_fault_box_nwcut_1000m_lcfar3000.msh**

Context (from PLAN_mesh_quality.md + locate_fault_slivers.py): the slivers sit in $z \in [-700, 0]$ m, distance-to-fault < 1.5 km — the faultfree-surface corridor. The gmsh-side fix (headroom clamp + lc_min floor + Netgen optimizer) already shipped and accepted 1–2 residual slivers at $\eta \approx 0.03$ – 0.09 . TMOP is a candidate polishing pass on those residuals.

Step 0 — measure and classify

```
cd miniapps/meshing
make mesh-quality mesh-optimizer pmesh-optimizer
./mesh-quality -m <...>/safs_fault_box_nwcut_1000m_lcfar3000.msh -size -aspr -skew -visit -no-vis
# also run mesh-optimizer once with -ni 0 (or read its banner) to print
# "Minimum det(J) of the original mesh is ..." -> sign decides metric choice.
```

If $\min \det(J) > 0$ (expected): use a shape metric + worst-case. If $\min \det(J) \leq 0$: switch to -mid 313 (untangling) first, then re-polish.

Step A — safe, invariant-preserving smoke test (**-fix-bnd**)

Pins every surface (fault, free surface, sides, bottom); only interior bulk nodes move. Cannot break any invariant; modest payoff but zero risk.

```
./mesh-optimizer -m <mesh> -o 1 -mid 303 -tid 3 -wctype 2 \
                  -fix-bnd -nor -ni 50 -li 50 -qo 4 -rtol 1e-5 -vl 1 -no-vis
# -mid 303 shape, -tid 3 keep grading, -wctype 2 PMean attacks worst tets,
# -nor essential at UTM scale.
```

Re-check with check_mesh_quality.py on the gmsh export (see Step C caveat).

Step B — the real fix: surface fitting (tangential relaxation)

Requires a small custom driver (clone pmesh-fitting.cpp) that: 1. loads the mesh, builds level sets s_{fault} (signed distance to the SAFS surface) and $s_{\text{free}} = z$; 2. marks fault DOFs (boundary attr 101) and free-surface DOFs (attr 102); 3. EnableSurfaceFitting(s_{fault} , fault_marker, c, eval) and EnableSurfaceFitting(s_{free} , free_marker, c, eval); 4. pins the box sides/bottom (attr 103/104) in their normal component; 5. metric 303/302, target -tid 3, -wctype 2, -nor, MPI via pmesh-optimizer for the 1.3M-tet size.

This lets corridor nodes slide on the fault and on $z=0$ (and along their intersection) — the configuration most likely to dilate the residual slivers.

Step C — round-trip back into the SAFS pipeline (integration caveat)

mesh-optimizer writes optimized.mesh in MFEM mesh format, not gmsh .msh. The SAFS driver (spatial_dyn_driver.cpp:802, Mesh smesh(cfg.mesh.path,1,1)) auto-detects format, and MFEM mesh format preserves boundary attributes (101–104), so the solver can consume the optimized mesh directly. But the project's QA + viz tooling (check_mesh_quality.py, msh_to_vtu.py) read gmsh .msh via meshio. So you must either (a) keep an MFEM-format copy for the solver and re-export a .msh for QA, or (b) extend the quality checker to read MFEM format. Verify after any TMOP run: `- mesh_zmax == 0` exactly

(free-surface invariant); - every fault triangle still shared by exactly 2 tets (embedding); - $\eta_{\min}$ improved, and no new $\eta < 0.1$ tets created elsewhere.

9. What TMOP will NOT solve (set expectations)

- No topology change. No edge swap / collapse / split, no vertex add/remove. If a sliver is a connectivity defect (4 nodes that are coplanar for any admissible position of the free nodes), TMOP cannot remove it. That is gmsh Netgen's job (`Mesh.OptimizeNetgen`, already on per Phase 3) or a local re-mesh.
 - Pinned-vertex slivers. A flat tet whose 4 vertices all sit on the fault-free-surface trace, with all 4 fixed, has no degrees of freedom left. Only surface fitting (which unpins them tangentially) or topology surgery helps.
 - It does not fix the root cause. Per `PLAN_mesh_quality.md` §Diagnosis, the slivers come from a ~ 1 m $\rightarrow$ 100 m vertical headroom pinch at the fault top. TMOP polishes symptoms; the geometric headroom clamp (Phase 1) is the cure.
 - Cost. 1.3M tets + worst-case Newton is non-trivial; use `pmesh-optimizer` under MPI, and consider `-pa/-d cuda`.
-

10. Recommendation

1. Diagnose first: run `mesh-quality + read mesh-optimizer's min det(J)` banner on the actual file. If nothing is inverted (likely), this is polish, not rescue.
2. Cheap try: Step A (`-fix-bnd -mid 303 -tid 3 -wctype 2 -nor`). If it removes the 1–2 residual slivers without violating invariants, done.
3. If Step A is too weak (expected for trace slivers): invest in the Step B surface-fitting driver — that is the only TMOP mode with the right DOFs for on-surface slivers.
4. If still stuck: it's topological — stay with the shipped gmsh route (headroom + `lc_min` + Netgen), or raise `--fault-top-clamp` toward the 250 m physics ceiling to give the mesher more room (cheaper and more reliable than any post-hoc node motion).

Either way, TMOP is complementary to the gmsh pipeline: gmsh fixes topology and respects Physical Surface constraints; TMOP smooths/untangles node positions and (via surface fitting) relaxes nodes tangentially on the fault and free surface. Neither alone is guaranteed to clear an on-trace sliver; together they cover both the topological and the geometric failure modes.

Open questions / to verify on the actual file

- Sign of **min det(J)** on `safs_fault_box_nwcut_1000m_lcfar3000.msh` — decides shape-metric vs 313. (Run the banner.)
- Does MFEM **Finalize** accept the interior fault as boundary elements without reordering in mesh-optimizer's new `Mesh(...)` path the same way the SAFS driver's loader does? Confirm the optimized mesh still classifies attr 101 as the fault when re-read by `WaveOperator`.
- Level-set source for fault fitting: the cleanest `s_fault` is a signed distance to the SAFS surface; check whether the velocity/stress sidecar pipeline already exposes one we can reuse rather than rebuild.
- Whether moving fault-trace nodes at all is acceptable physically — the fault is the data (Fuis SAFS geometry). Surface fitting keeps nodes on the fault but slides them along it; confirm that tangential resampling of the fault triangulation is acceptable to the on-fault stress/velocity projection step (it changes which fault triangle a given DOF lands in).